

Question 1– Content Review and Feedback Platform

Scenario

A marketing team frequently writes product descriptions and blog articles. They need a simple platform where users can submit content, receive automated feedback, and review previous submissions.

Your task is to build a system that stores content submissions and generates structured feedback for users.

Technology Requirements

Backend-Node.js + Express and MongoDB

Frontend-React

Functional Requirements

Authentication

- Implement JWT-based authentication.
- Users should be able to: Register, Login and Access protected routes

Content Submission

- Users can submit text content.
- Each submission should include: Title, ContentCategory and Created Timestamp

Feedback Generation

The system should generate feedback containing:

- Readability Score
- Clarity Score
- Suggested Improvements

The feedback can be generated using: An external AI API OR a custom rule-based implementation

Backend APIs

Authentication

- Register User and Login User

Content

- Submit Content
- Retrieve Submission History
- Retrieve Submission Details

Feedback

- Generate Feedback
- Retrieve Feedback

Frontend Dashboard

Build a dashboard containing:

- Login/Register pages
- Content Submission Form
- Submission History
- Feedback Details View
- Summary Cards
 - Total Submissions
 - Average Score
 - **Recent Reviews**

Create a dedicated service layer for feedback generation.

Requirements:

- Feedback logic must be implemented in a separate module (e.g., feedbackService.js).
- The application should not directly call the feedback logic from route handlers.
- The design should allow replacing the implementation later (e.g., switching from rule-based feedback to an AI API).

Example Data Model

Content Submission

```
{
  "id": "uuid-string",
  "title": "Product Description",
  "content": "This product helps users improve productivity.",
  "category": "Marketing",
  "feedback": {
    "readabilityScore": 82,
    "clarityScore": 78,
    "suggestions": [
      "Use shorter sentences",
      "Add more specific product benefits"
    ]
  }
}
```

+251912662951

Info@fineto.fi

www.fineto.fi

Old Airport Munteha Bldg 2nd floor
Addis Ababa ET



```

    },
    "createdAt": "2026-06-19T10:30:00Z"
  }

```

Deliverables

1. Source Cod-Provide the complete source code for both frontend and backend applications.

2. Documentatio-Provide a README.md containing:

- Installation instructions
- Environment variables
- Setup and execution steps

3. Design Summary-Provide a brief explanation of:

- Architecture decisions
- Service-layer implementation
- Feedback generation approach
- Trade-offs made

4. Diagrams-Include a simple architecture or data-flow diagram showing:

- React Frontend
- Express Backend
- MongoDB Database
- Feedback Service Layer

Question 2 – Team Task Management System

Scenario

A small software company needs an internal task management system to help teams organize work, track progress, and monitor task completion. The system should allow managers to assign tasks and team members to update their status.

Technology Requirements

Backend

- Node.js + Express
- MongoDB

Frontend

- React

Functional Requirements

Implement JWT-based authentication.

Users should be able to: Register, Login and Access protected routes

User Roles

Support two user roles: Admin and Team Member

Admins can: Create tasks and Assign tasks

Team Members can: View assigned tasks and Update task status

Task Management

Each task should contain:

- Title
- Description
- Priority (Low, Medium, High)
- Status (To Do, In Progress, Completed)
- Assigned User
- Due Date
- Created Timestamp

Store all data in MongoDB. **Backend APIs**

Authentication

- Register User
- Login User

Users

- Retrieve Users

Tasks

- Create Task
- Update Task
- Retrieve Tasks
- Retrieve Tasks by Assigned User

Frontend Dashboard

Build a dashboard containing:

- Login/Register pages
- Task List
- Task Details Page
- Task Creation Form
- Status Summary Cards
 - o Total Tasks
 - o Completed Tasks
 - o Pending Tasks

Implement file attachments for tasks.

Requirements:

- Users can upload a file when creating or updating a task.
- Store file metadata in MongoDB.
- Use Multer for file uploads.
- Display uploaded file information in the task details page.

Example Data Model

Task

```
{
  "id": "uuid-string",
  "title": "Create Login Page",
  "description": "Implement user authentication UI",
  "priority": "High",
  "status": "In Progress",
  "assignedTo": "user-id",
  "attachment": "design.pdf",
  "dueDate": "2026-06-30",
  "createdAt": "2026-06-19T10:30:00Z"
}
```

Deliverables

1. Source Code-Provide the complete source code for both frontend and backend applications.

2. Documentation

Provide a README.md containing:

- Installation instructions
- Environment variables
- Setup and execution steps

3. Design SummaryProvide a brief explanation of:

- Architecture decisions

- Role-based access implementation
- File upload handling
- Trade-offs made

4. Diagrams

Include a simple architecture or data-flow diagram showing:

- React Frontend
- Express Backend
- MongoDB Database
- File Upload Flow